Frequently Asked Questions

Setting up and running Julia



My Julia code does not run in Zed. What can I do?

The commands in **bold** are in Zed's command palette (`Ctrl+Shift+P`, macOS `Cmd+Shift+P`).

- **Nothing happens when I press `Ctrl+Shift+Enter`.** Save the file with the ending `.jl` and press the shortcut a second time: right after Zed starts, the first press can get lost while Zed looks for its kernels. Each cell has to start with `# %%` at the very beginning of a line.
- **Zed does not list the kernel `optimization-with-julia-1.13`.** Run **repl: refresh kernelspecs**. If it is still missing, repeat steps 3 to 5 of the first tutorial and look for error messages after `installkernel`.
- **A cell keeps running, but neither output nor an error appears, often right after you installed the kernel again.** Zed still uses the kernel as it was when Zed started. Quit Zed completely (macOS: `Cmd+Q`), start it again and choose `optimization-with-julia-1.13` in the kernel selector at the top right.
- `ArgumentError: Package JuMP not found in current path` (or any other package) in the output below a cell. Zed started another kernel, for example `julia-1.13`, which does not use the workshop environment. Check the kernel selector at the top right of the editor and choose `optimization-with-julia-1.13`. If the kernel is right, the package is not installed yet: run `using Pkg; Pkg.add("JuMP")` in a cell.
- **A package is `not found` although you installed it, and the kernel selector shows `optimization-with-julia-1.13`.** The `Pkg.add` ran in a different environment than the kernel: either in a plain Julia terminal without `julia --project=.` (the package went into your global environment) or, if you set up the workshop before 30 October 2026, in the workshop folder while your kernel still uses the old environment in the `optimization-workshop` subfolder. Quick fix: run `using Pkg; Pkg.add("JuMP")` in a Zed cell — a cell always installs into the environment the kernel uses. Run `println(Base.active_project())` in a cell to see that environment. If the path ends in `optimization-workshop/Project.toml`, move `Project.toml` and `Manifest.toml` from the subfolder into your workshop folder, close Zed and run steps 3 and 5 of the first tutorial again, so the kernel uses the workshop folder itself.
- `UndefVarError: ... not defined`. The cell uses a variable from a cell that has not run yet. Run the cells above it first. After **repl: restart**, all variables are gone and you start again from the top.
- **Plots do not appear.** First check the kernel selector at the top right: only `optimization-with-julia-1.13` shows plots, other kernels such as `julia-1.13` do not. If it is the right one, your kernel was installed without the `zed_plots` part of step 5 in the first tutorial: close Zed and run steps 3 and 5 again. Interactive plots made with `plotly()` cannot be shown in Zed; leave that line out.
- **A model keeps solving and does not stop. repl: interrupt** cannot stop a running solver. Use **repl: restart** instead and run your cells again from the top.

- **The kernel does not start after a Julia update.** The kernel still points to the old Julia version, which the update removed. Close Zed and run steps 3 and 5 of the first tutorial again.
- **The first cell takes minutes.** Julia compiles each package once on your computer, so the first `using JuMP` or `using Plots` is slow, on Windows even slower because Microsoft Defender scans the new files. Let it finish: afterwards, the same commands are fast.
- **An exclamation mark appears in Zed's title bar, and code completion does not work.** Zed opened your folder in Restricted Mode. Click the exclamation mark and trust your workshop folder (only that folder, not your whole user folder).
- **`Ctrl+C` does not copy in Zed's terminal (Windows, Linux).** In the terminal, `Ctrl+C` stops the running program; copy with `Ctrl+Shift+C`.
- **Code is all one color, the bottom right of Zed says "Plain Text", and `Ctrl+Shift+Enter` does nothing.** The Julia extension is missing. Open **zed: extensions**, search for "Julia", install the extension called Julia and open the file again.
- **The status bar shows "Downloading jetls…" or "Checking for updates to jetls…" for minutes.** The Julia extension installs its language server, JETLS, with your Julia: the first time, and again after some extension updates. This can take several minutes. Wait until the message disappears; running cells works in the meantime.
- **"Failed to run jetls".** Click the message to see why. JETLS needs Julia 1.12.2 to 1.13: if the message names an older version such as 1.10, remove that version (see the entries on old Julia versions below). If Zed cannot find `julia`, restart Zed. If it still fails, switch the language server off.
- **Red underlines in the editor, such as "Package JuMP not found in current path" or "`DataFrame` is not defined", although the cell runs fine.** If your `.jl` file lies outside your workshop folder, the language server does not find the workshop environment and its packages. Move your tutorial files, together with their `data` folders, into your workshop folder and open them again from there (keep your workshop folder open in Zed); the first check there can take a few minutes. Until then, trust the output below the cell and do not follow the underline's `Pkg.add` hint. If the underlines stay, switch the language server off.
- **Underlines in the test cells of a tutorial, such as "`scatter_plot` is not defined".** The test cells check variables that you only create when you solve the exercise. Step 5 of the first tutorial writes a settings file into your workshop folder that switches these underlines off: close Zed and run steps 3 and 5 again.
- **Zed or your laptop gets slow after a while, and a `julia` process uses gigabytes of memory.** The language server keeps growing while you edit. Run **editor: restart language server**; if it happens again, switch the language server off.
- **How do I switch the language server off?** Run **zed: open settings file**, add `"languages": { "Julia": { "enable_language_server": false } }` inside the outer curly braces (with a comma after the entry before it) and restart Zed. If the file already has a `"languages"` entry, add only `"Julia": { "enable_language_server": false }` inside it. Colors and running cells keep working; only completions and the underlines go away.

If Zed itself does not run on your computer, use the VS Code fallback described in the first tutorial. The Jupyter notebooks of this workshop run there as well.

Packages do not install or load. What can I do?

- **`could not download https://pkg.julialang.org/registries`, `failed to clone from https://github.com/JuliaRegistries/General.git` or `Unable to automatically download/install artifact`.** Julia cannot reach its package servers. Check that the internet works in your browser and try again, or switch to another network such as a phone hotspot. If it still fails, run `ENV["JULIA_PKG_SERVER"] = ""` in Julia before `Pkg.add(...)`: Pkg then downloads from GitHub instead.
- **`The following 1 package failed to precompile` or `Failed to precompile Plots`.** A package cache is broken, often after an interrupted first `using`. Close Zed and any other Julia window, delete the folder `.julia/compiled/v1.13` in your user folder (Windows: `%USERPROFILE%\.julia\compiled\v1.13`), start Julia in your workshop folder with `julia --project=.` and run `using Pkg; Pkg.precompile()`.
- **`Package JuMP [...] is required but does not seem to be installed`.** The workshop environment lists the package, but its files are missing, for example after you copied your workshop folder to another laptop. Start Julia in your workshop folder with `julia --project=.` and run `using Pkg; Pkg.instantiate()`.
- **`The active manifest file has dependencies that were resolved with a different julia version`.** You set up the workshop environment with another Julia version. Close Zed and check that `julia --version` shows 1.13. Then run steps 3 and 5 of the first tutorial again, with `using Pkg; Pkg.resolve()` in between.
- **`Unsatisfiable requirements detected for package`.** The package versions do not fit together, often because the package list on your laptop is out of date. Run `using Pkg; Pkg.update()` in the workshop environment. If the error names a package the workshop does not use, remove it with `Pkg.rm("Name")`.
- **`No space left on device`.** The `.julia` folder in your user folder grows by several GB over time. Run `using Pkg; Pkg.gc()` and delete the `compiled` folders of Julia versions you no longer use, such as `.julia/compiled/v1.10`.

Julia or Zed does not work on my Windows laptop. What can I do?

- **The terminal does not find `julia`.** Open a new terminal; if you use the terminal inside Zed, restart Zed. If that does not help, open *Settings → Apps → Advanced app settings → App execution aliases* and switch on `julia.exe` and `juliaup.exe`. If the Microsoft Store is blocked on your laptop, use the MSI installer instead.
- **"Program 'julia.exe' failed to run: The file cannot be accessed by the system".** An installation through the Microsoft Store, `winget` or the MSIX App Installer did not finish and left a broken alias for `julia.exe` behind. The same alias makes `;julia` inside Julia fail with `permission denied (EACCES)`. Uninstall *Julia* under *Settings → Apps → Installed*

apps. If you also installed a standalone version such as the 1.10 LTS, uninstall it too: the workshop runs Julia 1.13 through juliaup. If `julia` still shows the error, switch off `julia.exe` and `juliaup.exe` under *Settings → Apps → Advanced app settings → App execution aliases*. Then install Julia with the MSI installer and open a new terminal; if you use the terminal inside Zed, restart Zed. If you had already installed the workshop kernel, run steps 3 and 5 of the first tutorial again.

- **`Access is denied. (os error 5)` or `Failed to move extracted archive` while Julia installs itself.** Your antivirus scans the freshly unpacked files and blocks juliaup. Run the same command again (`julia` or `juliaup add release`): it usually works on the second or third try. Delete leftover `julia-temp-...` folders in `%USERPROFILE%\.julia\juliaup`. If you use an antivirus other than Microsoft Defender, add `%USERPROFILE%\.julia` as an exception.
- **`julia --version` shows an old version such as 1.10, or Zed offers the kernel `optimization-with-julia-1.10`.** Another Julia comes first, usually a standalone installation. Run `where.exe julia` in PowerShell to see all of them. Uninstall every "Julia 1.x.y" entry under *Settings → Apps → Installed apps* and keep the one called "Julia" (juliaup). Then run `juliaup add release` and `juliaup default release`, open a new terminal or restart Zed, and run steps 3 and 5 of the first tutorial again.
- **Installing from the Microsoft Store or with `winget` fails with an error code.** `0x80073D01` or "blocked by policy": your laptop does not allow the Store, so use the MSI installer. `0x80073CF4`: drive C: is full; free some space. `0x80073CF3` or `0x80073CFB`: an older Julia is in the way; uninstall *Julia* under *Settings → Apps → Installed apps* and try again.
- **`The Julia launcher failed to figure out which juliaup channel to use` or `Io error. Look inside err_kind for more details`.** Julia is installed, but no Julia version inside it, often because the laptop was offline at the first start. Check your internet connection and run `juliaup add release` and `juliaup default release`. If that fails, uninstall *Julia*, delete the folder `%USERPROFILE%\.julia\juliaup`, restart Windows and install Julia again.
- **"An Application Control policy has blocked this file", "Smart App Control has blocked part of this app" or error `0xc0e90002`.** Windows Smart App Control blocks the package files that Julia compiles on your laptop. Switch it off under *Windows Security → App & browser control → Smart App Control* and keep it off while you work with Julia: switched on again, it can block every package Julia compiles later. Then delete the folder `%USERPROFILE%\.julia\compiled\v1.13` and run `using Pkg; Pkg.precompile()` in the workshop environment. On a laptop managed by your university or employer, only the IT department can allow these files; until then, start Julia in your workshop folder with `julia --pkgimages=no --project=.` in the first tutorial and add `"--pkgimages=no"` right after the `--project` argument of its `installkernel` call. Packages then load more slowly, but they load.
- **Precompiling IJulia fails with an `EPIPE` error.** Run `using IJulia` again: the error is known on Windows and usually disappears on the second attempt.
- **Your laptop has a Snapdragon (ARM) processor.** Julia runs there in x64 emulation: the workshop setup should work the same way, only slower.

- **Your workshop folder lies in OneDrive.** Synced folders can block Julia from replacing files. Use a plain folder such as `C:\Users\<name>\julia-workshop`.

Julia does not work on my Mac or Linux laptop. What can I do?

- **`zsh: command not found: julia` or `bash: julia: command not found` right after the install.** The terminal was open before the install and has not read the new settings. Close all terminal windows and Zed, then open a new terminal. If that does not help, run `. ~/.zshrc` (macOS) or `. ~/.bashrc` (Linux). If `juliaup` works but `julia` does not, run `juliaup self update`; if that does not help, run the install command again.
- **`julia --version` shows another version than 1.13.** An older Julia from Homebrew or a `.dmg` download comes first. Run `which -a julia` to see all of them. Keep juliaup's: `~/.juliaup/bin/julia`, or `/opt/homebrew/bin/julia` if you installed juliaup with `brew install juliaup`. Remove the others, for example with `brew uninstall julia`, by moving `Julia-1.x.app` from *Applications* to the Trash, or with `sudo rm /usr/local/bin/julia` if that path is listed. Then open a new terminal and run steps 3 and 5 of the first tutorial again.
- **Ubuntu suggests `sudo snap install julia` or `sudo apt install julia`.** Don't: these packages are outdated. Use the install command from julialang.org/downloads. If it stops with `curl: (23) Failure writing output to destination`, your `curl` comes from snap: run `sudo snap remove curl` and `sudo apt install curl`, then run the install command again.
- **`Permission denied (os error 13)` while installing or starting Julia.** Files in your user folder belong to the administrator, usually because Julia once ran with `sudo`. Run `sudo chown -R "$(whoami)" ~/.julia ~/.juliaup`, then start Julia without `sudo`. Never use `sudo` for `julia` or `Pkg`.

Still stuck? Ask the course chatbot on this website: it knows these steps.